

Finite element methods for the wave equation

Mathematical derivation and implementation analysis

Contents

1	Introduction and mathematical problem	3
2	Weak formulation	4
3	Galerkin spatial discretization	5
3.1	Discontinuous Galerkin space and face terms	6
4	Time discretization	7
4.1	Theta scheme for displacement and velocity	7
4.2	Gauss-Lobatto quadrature and diagonal mass matrix	8
4.3	Explicit centered scheme and leapfrog interpretation	8
4.4	Initialization and discrete wave energy	9
5	Implementation	9
5.1	Assembled theta solver	9
5.1.1	Implementation in the time loop	10
5.2	Matrix-free formulation and implementation	10
5.2.1	Continuous Galerkin implementation	11
5.2.2	Discontinuous Galerkin implementation	11
5.3	Energy conservation analysis	12
5.3.1	Theoretical energy of the Gaussian	12
5.3.2	Energy computed by the theta solver	12
5.3.3	Natural and staggered energy for matrix-free CG	13
5.3.4	Natural and staggered energy for DG/SIPG	13
5.3.5	Error used in the analysis	14
5.3.6	Summary of the numerical results	14
5.4	Convergence analysis using a manufactured solution	15
5.4.1	Manufactured problem	15
5.4.2	Discretizations and refinement procedure	16
5.4.3	Numerical results	16
6	Numerical dispersion analysis	17

6.1	Purpose and experimental configuration	17
6.2	Reference solution and peak measurement	18
6.3	Diagnostic quantities	18
6.4	Polynomial-degree study	19
6.5	Matrix-free CG and DG comparison	19
7	Benchmark performance comparison	19
7.1	Experimental configuration and timing definition	19
7.2	Measured performance	20
7.3	Interpretation of the runtime differences	20
8	Parallel performance and scaling on MeluXina	21
8.1	Computing environment and numerical configuration	21
8.2	Definitions of the performance metrics	22
8.3	Strong scaling for 2D problem	22
8.4	Weak scaling in two spatial dimensions	23
8.5	Scaling results and solver comparison	25

1 Introduction and mathematical problem

This report presents the mathematical formulation and implementation of three finite element solvers for the wave equation, developed using the deal.II library. The project considers a continuous Galerkin solver with theta-method time integration, a matrix-free continuous Galerkin solver, and a matrix-free discontinuous Galerkin solver based on the symmetric interior penalty method. The two matrix-free solvers use explicit centered time integration.

The presentation follows the progression from the continuous mathematical problem to its numerical implementation. First, the wave equation is introduced together with the boundary and initial conditions. The weak formulation is then derived by integration by parts, and the Galerkin approximation is used to obtain the semi-discrete equations. This leads to the definition of the mass and stiffness matrices. For the discontinuous Galerkin method, additional interface terms are introduced to connect neighboring elements and impose boundary conditions weakly.

The report subsequently develops the time-discrete equations for the theta method and the explicit centered scheme. Particular attention is given to Gauss–Lobatto quadrature, diagonal mass approximation and the stability requirements of explicit integration. The implementation section then explains how these equations are evaluated through assembled matrices or matrix-free cell and face operations.

Finally, the manufactured-solution verification procedure and the benchmark design are discussed. Throughout the report, theoretical properties are distinguished from observations based on source inspection. The objective is to explain how the mathematical choices determine the structure and computational requirements of each solver.

$$\begin{cases} \frac{\partial^2 u}{\partial t^2} - \Delta u = f & \text{in } \Omega \times (0, T), \\ u = 0 & \text{on } \partial\Omega \times (0, T), \\ u(\mathbf{x}, 0) = u_0(\mathbf{x}) & \text{in } \Omega, \\ \frac{\partial u}{\partial t}(\mathbf{x}, 0) = v_0(\mathbf{x}) & \text{in } \Omega. \end{cases} \quad (1)$$

- **Classification.** The wave equation is a *linear, second-order, hyperbolic* partial differential equation. It is linear because the unknown and its derivatives appear linearly, and second-order because the highest derivatives have order two. Its hyperbolic character describes wave propagation at finite speed, normalized here to $c = 1$.
- **Unknown field** $u(\mathbf{x}, t)$. Represents the displacement or wave amplitude at spatial position $\mathbf{x}$ and time t .
- **Temporal acceleration** $\frac{\partial^2 u}{\partial t^2}$. Describes the rate of change of the velocity of the field.
- **Spatial Laplacian** Δu . Measures spatial curvature and couples neighbouring points, driving wave propagation. In d spatial dimensions,

$$\Delta u = \sum_{i=1}^d \frac{\partial^2 u}{\partial x_i^2}.$$

- **Forcing term** $f(\mathbf{x}, t)$. Represents an external source exciting the wave. The PDE is homogeneous when $f = 0$ and nonhomogeneous otherwise. $\Omega \subset \mathbb{R}^d$ during the interval $0 < t < T$, where T is the final simulation time.
- **Boundary condition** $u = 0$ on $\partial\Omega \times (0, T)$. This homogeneous Dirichlet condition fixes the displacement to zero along the boundary $\partial\Omega$, corresponding to a fixed boundary in a displacement model.

- **Initial displacement** $u(\mathbf{x}, 0) = u_0(\mathbf{x})$. Specifies the initial shape of the wave.
- **Initial velocity** $\frac{\partial u}{\partial t}(\mathbf{x}, 0) = v_0(\mathbf{x})$. Specifies the initial motion of the wave. Both initial displacement and initial velocity are required because the equation is second-order in time.

The unknown u represents displacement. Because the equation is second order in time, both an initial displacement and an initial velocity must be prescribed. The boundary condition fixes the displacement to zero.

2 Weak formulation

Let $V = H_0^1(\Omega)$.

$$V = H_0^1(\Omega) = \left\{ v \in H^1(\Omega) \mid v = 0 \text{ on } \partial\Omega \right\} \quad (2)$$

The zero trace condition reflects the homogeneous Dirichlet boundary condition prescribed for u .

Multiplying the wave equation by a test function $q \in V$ and integrating over Ω , we obtain

$$\int_{\Omega} \frac{\partial^2 u}{\partial t^2} q \, dx - \int_{\Omega} \Delta u \, q \, dx = \int_{\Omega} f q \, dx.$$

Integrating the Laplacian term by parts gives

$$- \int_{\Omega} \Delta u \, q \, dx = \int_{\Omega} \nabla u \cdot \nabla q \, dx - \int_{\partial\Omega} \frac{\partial u}{\partial n} q \, d\sigma.$$

The boundary integral vanishes because $q = 0$ on $\partial\Omega$.

For sufficiently regular solutions, the weak formulation reads: for all $t \in (0, T)$, find $u(t) \in V$ such that $u(0) = u_0$, $\partial_t u(0) = v_0$, and

$$\int_{\Omega} \frac{\partial^2 u}{\partial t^2} q \, dx + \underbrace{\int_{\Omega} \nabla u \cdot \nabla q \, dx}_{a(u, q)} = \underbrace{\int_{\Omega} f(\mathbf{x}, t) q \, dx}_{F(q; t)} \quad \text{for all } q \in V. \quad (3)$$

Here, $a : V \times V \rightarrow \mathbb{R}$ is the bilinear form associated with the spatial differential operator, while $F(\cdot; t) : V \rightarrow \mathbb{R}$ is the linear functional associated with the forcing at time t . The notation $u(t)$ denotes the spatial function $\mathbf{x} \mapsto u(\mathbf{x}, t)$.

To prepare the time discretization, introduce the velocity $v = \partial_t u$. For sufficiently regular solutions, the wave equation can then be written as the first-order variational system

$$\begin{cases} \int_{\Omega} \frac{\partial u}{\partial t} q \, dx = \int_{\Omega} v q \, dx \\ \int_{\Omega} \frac{\partial v}{\partial t} q \, dx + a(u, q) = F(q; t) \end{cases} \quad \text{for all } q \in V, \quad (4)$$

with initial conditions $u(0) = u_0$ and $v(0) = v_0$.

For sufficiently regular solutions, if we multiply by $v = \frac{\partial u}{\partial t}$ in the weak formulation we find that

$$\int_{\Omega} \frac{\partial^2 u}{\partial t^2} \frac{\partial u}{\partial t} \, dx + \int_{\Omega} \nabla u \cdot \nabla \frac{\partial u}{\partial t} \, dx = \int_{\Omega} f \frac{\partial u}{\partial t} \, dx.$$

Using the identities

$$\int_{\Omega} \frac{\partial^2 u}{\partial t^2} \frac{\partial u}{\partial t} \, dx = \frac{d}{dt} \left(\frac{1}{2} \left\| \frac{\partial u}{\partial t} \right\|_{L^2(\Omega)}^2 \right), \quad \int_{\Omega} \nabla u \cdot \nabla \frac{\partial u}{\partial t} \, dx = \frac{d}{dt} \left(\frac{1}{2} \left\| \nabla u \right\|_{L^2(\Omega)}^2 \right),$$

we obtain the energy balance

$$E(t) = \frac{1}{2} \left\| \frac{\partial u}{\partial t} \right\|_{L^2(\Omega)}^2 + \frac{1}{2} \left\| \nabla u(t) \right\|_{L^2(\Omega)}^2, \quad \frac{dE}{dt} = F\left(\frac{\partial u}{\partial t}; t\right) = \int_{\Omega} f \frac{\partial u}{\partial t} dx. \quad (5)$$

The two terms in $E(t)$ represent kinetic and elastic energy, respectively. When $f = 0$, the total wave energy is conserved: $E(t) = E(0)$.

3 Galerkin spatial discretization

Let $\mathcal{T}_h = \{K_c\}_{c=1}^{N_{\text{el}}}$ be a mesh of Ω . Define

$$X_h^r(\Omega) = \left\{ v_h \in C^0(\overline{\Omega}) : v_h|_{K_c} \in \mathbb{Q}_r(K_c), \quad c = 1, \dots, N_{\text{el}} \right\},$$

where, $\mathbb{Q}_r(K_c)$ denotes the tensor-product polynomial space of degree at most r in each spatial coordinate:

$$\mathbb{Q}_r(K_c) = \text{span} \left\{ x_1^{\alpha_1} \cdots x_d^{\alpha_d} : 0 \leq \alpha_i \leq r, \quad \alpha_i \in \mathbb{N}_0, \quad i = 1, \dots, d \right\}.$$

The finite element space satisfying the homogeneous Dirichlet boundary condition is then

$$V_h = X_h^r(\Omega) \cap H_0^1(\Omega).$$

the space of continuous finite element functions whose restriction to each reference cell is a tensor-product polynomial of degree at most r in each coordinate. This corresponds to the `FE_Q` elements used in the conforming implementations.

The semi-discrete Galerkin problem is: find $u_h(t) \in V_h$ such that

$$\int_{\Omega} \frac{\partial^2 u_h}{\partial t^2} q_h dx + a(u_h, q_h) = F(t; q_h) \quad \forall q_h \in V_h, \quad (6)$$

with $u_h(0) = u_{0,h}$ and $\partial_t u_h(0) = v_{0,h}$.

Let $\{\varphi_j\}_{j=1}^{N_h}$ be a basis be the Lagrangian basis functions of the space V_h . Write

$$u_h(\mathbf{x}, t) = \sum_{j=1}^{N_h} U_j(t) \varphi_j(\mathbf{x}).$$

Testing (6) with φ_i gives

$$\sum_j \left(\int_{\Omega} \varphi_j \varphi_i \right) \ddot{U}_j + \sum_j \left(\int_{\Omega} \nabla \varphi_j \cdot \nabla \varphi_i \right) U_j = \int_{\Omega} f \varphi_i.$$

Define the mass matrix, stiffness matrix and load vector by

$$\begin{aligned} M_{ij} &= \int_{\Omega} \varphi_j \varphi_i, & A_{ij} &= \int_{\Omega} \nabla \varphi_j \cdot \nabla \varphi_i, \\ F_i(t) &= \int_{\Omega} f(t) \varphi_i. \end{aligned} \quad (7)$$

The resulting system of ordinary differential equations is

$$M \ddot{\mathbf{U}} + \mathbf{A} \mathbf{U} = \mathbf{F}(t), \quad \mathbf{U}(0) = \mathbf{U}_0, \quad \dot{\mathbf{U}}(0) = \mathbf{V}_0. \quad (8)$$

Equivalently, introducing the velocity coefficient vector $\mathbf{V}(t) = \dot{\mathbf{U}}(t)$, we obtain the first-order system

$$\begin{cases} M\dot{\mathbf{U}}(t) = M\mathbf{V}(t), \\ M\dot{\mathbf{V}}(t) + A\mathbf{U}(t) = \mathbf{F}(t), \end{cases} \quad \begin{cases} \mathbf{U}(0) = \mathbf{U}_0, \\ \mathbf{V}(0) = \mathbf{V}_0. \end{cases} \quad (9)$$

Here, $\mathbf{V}(t) \in \mathbb{R}^{N_h}$ contains the coefficients of the discrete velocity field. Since M is invertible on the independent degrees of freedom, the first equation implies $\dot{\mathbf{U}} = \mathbf{V}$. Substituting this identity into the second equation recovers (8).

The matrices are symmetric and positive definite on the independent Dirichlet degrees of freedom. The assembled solver evaluates (7) with tensor-product Gaussian quadrature using $p+1$ points per direction. The matrix-free solvers instead use Gauss-Lobatto quadrature and a corresponding diagonal mass approximation, as explained below.

3.1 Discontinuous Galerkin space and face terms

The third solver uses the discontinuous space

$$V_h^{DG} = \left\{ v_h \in L^2(\Omega) : v_h|_{K_c} \in \mathbb{Q}_r(K_c), \quad c = 1, \dots, N_{\text{el}} \right\}, \quad r = 4. \quad (10)$$

Functions may jump across cell faces, so elementwise integration by parts requires numerical face terms. For an interior face shared by K^- and K^+ , choose $\mathbf{n}$ pointing from K^- to K^+ and define

$$[v] = v^- - v^+, \quad \{\nabla v\} = \frac{1}{2}(\nabla v^- + \nabla v^+).$$

The symmetric interior penalty form corresponding to the implemented fluxes is

$$\begin{aligned} a_{DG}(u, v) = & \sum_K \int_K \nabla u \cdot \nabla v \\ & - \sum_{F \in \mathcal{F}_h^i} \int_F (\{\nabla u\} \cdot \mathbf{n}[v] + \{\nabla v\} \cdot \mathbf{n}[u]) \\ & + \sum_{F \in \mathcal{F}_h^i} \int_F \sigma[u][v] \\ & - \sum_{F \in \mathcal{F}_h^b} \int_F (\partial_n u v + \partial_n v u) + \sum_{F \in \mathcal{F}_h^b} \int_F 2\sigma u v. \end{aligned} \quad (11)$$

The interior terms provide consistency, symmetry and penalization of jumps. The boundary terms impose $u = 0$ weakly. The factor 2σ at boundary faces is intentional in this expression: it reproduces the actual boundary flux in the source.

The penalty parameter is

$$\sigma = \frac{1.5(p+1)(p+d)}{dh}. \quad (12)$$

For $p = 4$, this is $22.5/h$ in two dimensions and $17.5/h$ in three dimensions.

The first interior consistency term replaces the normal derivative obtained from elementwise integration by parts with an average flux. Because a discontinuous test function has two traces, the adjacent cell contributions combine with its jump. The second consistency term is the transpose contribution, making the bilinear form symmetric; the positive jump penalty counteracts the

potentially negative consistency terms. On a boundary face, the exterior condition is supplied through the weak Dirichlet terms instead of eliminating nodal unknowns.

For a sufficiently regular exact solution, its jump vanishes across interior faces and its trace vanishes on the boundary. Substituting that solution into the form therefore recovers the elementwise weak equation.

4 Time discretization

4.1 Theta scheme for displacement and velocity

Let $t_n = nk$, where $k = \Delta t$, and define

$$\mathbf{F}^{n+\theta} = \theta \mathbf{F}^{n+1} + (1 - \theta) \mathbf{F}^n.$$

Applying the theta method to both equations in (9) gives

$$\left\{ \begin{array}{l} M \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{k} = M [\theta \mathbf{V}^{n+1} + (1 - \theta) \mathbf{V}^n] \end{array} \right. \quad (13a)$$

$$\left\{ \begin{array}{l} M \frac{\mathbf{V}^{n+1} - \mathbf{V}^n}{k} + A [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] = \mathbf{F}^{n+\theta} \end{array} \right. \quad (13b)$$

Equation (13a) implies

$$M \mathbf{V}^{n+1} = M \mathbf{V}^n - kA [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] + k \mathbf{F}^{n+\theta}.$$

Substituting this expression into (13b) eliminates the new velocity. The displacement is obtained by solving

$$\boxed{(M + \theta^2 k^2 A) \mathbf{U}^{n+1} = [M - \theta(1 - \theta) k^2 A] \mathbf{U}^n + k M \mathbf{V}^n + \theta k^2 \mathbf{F}^{n+\theta}.} \quad (14)$$

The velocity is then recovered from

$$\boxed{M \mathbf{V}^{n+1} = M \mathbf{V}^n - kA [\theta \mathbf{U}^{n+1} + (1 - \theta) \mathbf{U}^n] + k \mathbf{F}^{n+\theta}.} \quad (15)$$

These are the two systems implemented in `WaveSolverTheta::run`. The default is $\theta = 1/2$, namely Crank-Nicolson applied to the first-order displacement-velocity system. It has second-order temporal accuracy for smooth solutions. For $\theta \neq 1/2$, the general theta method is normally first order in time.

For a homogeneous semi-discrete eigenmode with frequency ω , the amplification factor has modulus

$$|g|^2 = \frac{1 + (1 - \theta)^2 k^2 \omega^2}{1 + \theta^2 k^2 \omega^2}. \quad (16)$$

Thus $\theta \geq 1/2$ gives non-amplifying modes, $\theta = 1/2$ preserves their amplitude, and $\theta < 1/2$ amplifies nonzero oscillatory modes. In particular, forward Euler is unstable for the undamped wave system for every positive time step.

For $\theta = 1/2$, exact linear solves preserve the quadratic discrete energy

$$E_h^n = \frac{1}{2} (\mathbf{V}^n)^T M \mathbf{V}^n + \frac{1}{2} (\mathbf{U}^n)^T A \mathbf{U}^n.$$

This follows by testing the two midpoint equations with the corresponding midpoint variables. Iterative solution tolerances can introduce energy errors. Unconditional stability also does not remove the need to resolve oscillations accurately in time.

4.2 Gauss-Lobatto quadrature and diagonal mass matrix

The second solver uses continuous $\mathbb{Q}_4$ finite elements. The finite element nodes and the quadrature points are chosen to be the same Gauss-Lobatto points. For degree $r = 4$, there are five points in each coordinate direction.

The advantage of this choice is that the mass matrix computed by quadrature is diagonal. Therefore, at every quadrature point, the product of two different basis functions is zero.

Using Gauss-Lobatto quadrature, the mass matrix entries are approximated by

$$(M_L)_{ij} = \sum_{K \in \mathcal{T}_h} \sum_q \omega_q |\det J_K(\hat{\mathbf{x}}_q)| \varphi_j(\mathbf{x}_{Kq}) \varphi_i(\mathbf{x}_{Kq}), \quad (M_L)_{ij} = 0 \quad \text{if } i \neq j. \quad (17)$$

Here, ω_q are the reference quadrature weights, J_K is the Jacobian of the element mapping, and $\mathbf{x}_{Kq}$ are the quadrature points on the physical element K .

On the uniform mesh used in the benchmark, assembling these element contributions preserves the diagonal structure. The resulting matrix M_L is an approximation of the consistent mass matrix M , rather than an exact evaluation of its integrals.

Because M_L is diagonal, applying its inverse requires only dividing each vector entry by the corresponding diagonal entry:

$$(M_L^{-1} \mathbf{b})_i = \frac{b_i}{(M_L)_{ii}}.$$

The inverse diagonal is computed once and reused during the time integration, avoiding a coupled mass-matrix solve at every step.

4.3 Explicit centered scheme and leapfrog interpretation

For the homogeneous wave equation, the second time derivative is approximated by a centered difference:

$$M_L \frac{\mathbf{U}^{n+1} - 2\mathbf{U}^n + \mathbf{U}^{n-1}}{k^2} + A_Q \mathbf{U}^n = 0,$$

where k is the time-step size and A_Q (A_{DGQ} for Discontinuous Galerkin) is the stiffness operator evaluated using Gauss-Lobatto quadrature. Rearranging gives the explicit update

$$\boxed{\mathbf{U}^{n+1} = 2\mathbf{U}^n - \mathbf{U}^{n-1} - k^2 M_L^{-1} A_Q \mathbf{U}^n.} \quad (18)$$

This three-level recurrence is the displacement form of leapfrog, also called the centered-difference scheme. To see the connection, define the derived half-step velocity $\mathbf{V}^{n+1/2} = (\mathbf{U}^{n+1} - \mathbf{U}^n)/k$. Equation (18) implies

$$\begin{cases} \mathbf{V}^{n+1/2} = \mathbf{V}^{n-1/2} - k M_L^{-1} A_Q \mathbf{U}^n, \\ \mathbf{U}^{n+1} = \mathbf{U}^n + k \mathbf{V}^{n+1/2}. \end{cases} \quad (19)$$

Velocity and displacement are then interpreted at staggered times. Equation (19) explains the name; the implementation stores the displacement vectors of (18), without independent half-step velocity vectors.

In the matrix-free implementation, A_Q is not assembled or stored. Its action on $\mathbf{U}^n$ is computed directly through finite element integration on each cell. The result is then multiplied by the inverse mass diagonal.

The centered time-stepping method is second-order accurate when a sufficiently accurate initialization is used. Unlike the implicit Crank–Nicolson method, it requires a sufficiently small time step. A modal stability requirement with a strict margin is

$$k^2 \lambda_{\max}(M_L^{-1} A_Q) < 4, \quad (20)$$

where $\lambda_{\max}$ is the largest eigenvalue of the discrete spatial operator.

The code selects a provisional time step proportional to the cell size:

$$k_0 = \frac{0.1}{r} h = 0.025h, \quad r = 4,$$

where h is the cell side length on the uniform mesh. This is a prescribed CFL-based choice; the code does not compute the largest eigenvalue to verify (20).

4.4 Initialization and discrete wave energy

Unlike the first-order displacement-velocity scheme, the centered recurrence requires two displacement levels before the main loop begins. Given the actual selected step k , one may compute an acceleration $\mathbf{a}^0 = M_L^{-1}(\mathbf{F}^0 - A_Q \mathbf{U}^0)$ and initialize $\mathbf{U}^1 = \mathbf{U}^0 + k \mathbf{V}^0 + k^2 \mathbf{a}^0/2$. An equivalent backward Taylor value initializes $\mathbf{U}^{-1}$.

For a symmetric positive stiffness operator A_* and diagonal positive mass M_* , the homogeneous recurrence preserves the staggered quantity

$$\mathcal{E}^{n+1/2} = \frac{1}{2} \left\| \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{k} \right\|_{M_*}^2 + \frac{1}{2} (\mathbf{U}^{n+1})^T A_* \mathbf{U}^n,$$

where $\|\mathbf{z}\|_B^2 = \mathbf{z}^T B \mathbf{z}$. Multiplication of the recurrence by $(\mathbf{U}^{n+1} - \mathbf{U}^{n-1})/2$ proves that the difference between consecutive staggered energies is zero. With $\mathbf{w} = (\mathbf{U}^{n+1} - \mathbf{U}^n)/k$ and $\bar{\mathbf{U}} = (\mathbf{U}^{n+1} + \mathbf{U}^n)/2$, the same energy is

$$\mathcal{E}^{n+1/2} = \frac{1}{2} \|\mathbf{w}\|_{M_* - k^2 A_*/4}^2 + \frac{1}{2} \|\bar{\mathbf{U}}\|_{A_*}^2.$$

This expression connects positivity of the energy to the spectral step restriction. At the limiting eigenvalue the first quadratic form becomes singular, which is another reason to keep a strict stability margin. The formula applies to the CG and DG recurrences when their respective stiffness operators satisfy the stated assumptions. It does not certify the current startup or the chosen CFL constants.

5 Implementation

5.1 Assembled theta solver

`setup_system` distributes degrees of freedom, constructs hanging-node and zero Dirichlet constraints, builds a distributed sparsity pattern and allocates Trilinos matrices and vectors. `assemble_matrices` constructs M and A once. Initial displacement and velocity are obtained by constrained L^2 projection using `VectorTools::project`.

At each time step, the code assembles the weighted load, constructs the right-hand sides of (14)-(15), copies and combines the constant matrices, applies zero boundary values and solves the two systems. Both solves use conjugate gradients with Jacobi preconditioning, at most 2000 iterations and a residual threshold of 10^{-8} times the right-hand-side norm. Constraints are distributed after each solve.

Although M and A are assembled once, the system matrices and preconditioners are reconstructed inside the time loop. The chosen step is half the minimum cell diameter.

5.1.1 Implementation in the time loop

Both matrix-free solvers maintain three vectors: `solution_`, `old_solution_` and `old_old_solution_`. Before each operator application, the code exchanges their contents using

```
old_old_solution_.swap(old_solution_);
old_solution_.swap(solution_);
```

After these exchanges, `old_solution_` contains $\mathbf{U}^n$ and `old_old_solution_` contains $\mathbf{U}^{n-1}$. The vector `solution_` is used as the destination for $\mathbf{U}^{n+1}$.

The wave operator combines the quadrature mass contribution $M_L(2\mathbf{U}^n - \mathbf{U}^{n-1})$ with the stiffness contribution $-k^2 A_Q \mathbf{U}^n$. It then applies the inverse mass diagonal. For DG, the stiffness contribution additionally includes the interior-face and boundary-face integrals.

Vector swaps rotate the displacement history without copying the complete vectors. Following the update, the CG solver distributes the constraints to enforce the prescribed boundary values.

5.2 Matrix-free formulation and implementation

The matrix-free approach evaluates the action of a finite element operator without assembling and storing its global matrix. The underlying spatial formulation remains unchanged: the operator is applied through local quadrature operations rather than a sparse matrix–vector multiplication.

To illustrate the principle, consider the stiffness matrix A and the finite element expansion

$$u_h(\mathbf{x}, t) = \sum_{j=1}^{N_h} U_j(t) \varphi_j(\mathbf{x}).$$

The action of A on the coefficient vector can be rewritten as

$$\begin{aligned} (AU)_i &= \sum_{j=1}^{N_h} U_j \int_{\Omega} \nabla \varphi_j \cdot \nabla \varphi_i \, dx \\ &= \sum_{c=1}^{N_{el}} \int_{K_c} \nabla u_h \cdot \nabla \varphi_i \, dx. \end{aligned} \tag{21}$$

Thus, the product can be computed directly by evaluating ∇u_h and integrating its scalar product with the test-function gradients on each element. Neither the global stiffness matrix nor the individual element matrices need to be formed.

For a fixed mesh, finite element space, quadrature rule and treatment of constraints, this evaluation is algebraically equivalent to multiplication by the corresponding assembled matrix, up to floating-point rounding. In the present project, the matrix-free solvers use Gauss–Lobatto quadrature, whereas the assembled theta solver uses Gaussian quadrature.

5.2.1 Continuous Galerkin implementation

The continuous matrix-free solver uses $\mathbb{Q}_4$ elements and the diagonal mass approximation M_L introduced previously. For the homogeneous wave equation, the explicit time update is evaluated in two stages:

$$\begin{cases} \mathbf{b}^n = M_L(2\mathbf{U}^n - \mathbf{U}^{n-1}) - k^2 A_Q \mathbf{U}^n, \\ \mathbf{U}^{n+1} = M_L^{-1} \mathbf{b}^n, \end{cases} \quad (22)$$

where A_Q denotes the stiffness operator evaluated using Gauss–Lobatto quadrature.

The mass and stiffness contributions to $\mathbf{b}^n$ are combined in a single cell loop. On each element, the implementation reads the coefficients of the current and previous displacements and evaluates u_h^n , u_h^{n-1} and ∇u_h^n at the quadrature points. It then integrates the expression

$$(2u_h^n - u_h^{n-1})\varphi_i - k^2 \nabla u_h^n \cdot \nabla \varphi_i$$

using the prescribed quadrature rule and adds the element contribution to the global vector.

These operations are implemented in `WaveOperationCG::local_apply` through the `FEEvaluation` class. The methods `read_dof_values` and `evaluate` provide the local values and gradients. The methods `submit_value`, `submit_gradient` and `integrate` construct the contributions, which are accumulated through `distribute_local_to_global`.

The method `WaveOperationCG::apply` executes the cell loop and applies the inverse mass diagonal. Since M_L is diagonal, the second stage of (22) reduces to

$$U_i^{n+1} = \frac{b_i^n}{(M_L)_{ii}}.$$

The inverse diagonal is computed once when the operator is initialized and reused throughout the time integration. Consequently, each time step requires no coupled linear solve.

The displacement history is maintained using three vectors. Their roles are exchanged by vector swaps, avoiding full copies of the solution arrays. The homogeneous Dirichlet constraints are distributed after each update.

5.2.2 Discontinuous Galerkin implementation

The DG solver uses `FE_DGQArbitraryNodes` with degree $p = 4$ and Gauss–Lobatto support points. Each element owns $(p + 1)^d$ independent degrees of freedom, including separate unknowns at geometrically coincident points on neighbouring elements. Continuity is therefore imposed through numerical fluxes rather than shared nodal values. In `WaveSolverDG::setup`, the constraint set is empty, since Dirichlet conditions are enforced weakly.

The matrix-free operator implements the SIPG form in Equation (11), using the penalty parameter defined in Equation (12). Its application combines three contributions. The cell routine, `local_apply`, uses `FEEvaluation` to integrate the displacement-history terms and the volume gradient contribution. The interior-face routine, `local_apply_face`, uses two `FEFaceEvaluation` objects to read the traces from neighbouring elements. Their displacement jump and average gradient provide the consistency, symmetry and penalty contributions, which are accumulated on both sides. Finally, `local_apply_boundary_face` applies the weak Dirichlet terms, including the boundary penalty 2σ .

The method `WaveOperationDG::apply` executes these three routines through `MatrixFree::loop`. For the undamped, unforced problem, their combined result is

$$\mathbf{b}^n = M_L(2\mathbf{U}^n - \mathbf{U}^{n-1}) - k^2 A_{DG,Q} \mathbf{U}^n,$$

where $A_{DG,Q}$ denotes the quadrature-based SIPG operator. The Gauss–Lobatto nodal quadrature produces a diagonal mass matrix, whose inverse is computed once when the operator is constructed. An entrywise multiplication then gives $\mathbf{U}^{n+1} = M_L^{-1} \mathbf{b}^n$, implementing the centered update in Equation (18) with the DG spatial operator. No global stiffness matrix or iterative linear solve is required. Three displacement vectors are rotated by swaps to retain the time history without copying their entries.

5.3 Energy conservation analysis

Energy monitoring provides a global consistency test for the three wave-equation solvers. It does not replace the manufactured-solution study, because a method may conserve a discrete energy while still exhibiting spatial, phase or dispersion errors. Nevertheless, for the undamped and unforced problem it is a useful diagnostic of stability and of the consistency between the spatial operator and the time integrator.

The benchmark solves

$$u_{tt} - \Delta u = 0 \quad \text{in } \Omega = [-15, 15]^d, \quad (23)$$

with homogeneous boundary conditions, zero initial velocity and the unit-width Gaussian initial displacement

$$u_0(\mathbf{x}) = \exp\left(-\frac{|\mathbf{x}|^2}{2}\right), \quad v_0(\mathbf{x}) = 0. \quad (24)$$

For the continuous problem, the energy defined in (5) satisfies

$$E(t) = \frac{1}{2} \|u_t(t)\|_{L^2(\Omega)}^2 + \frac{1}{2} \|\nabla u(t)\|_{L^2(\Omega)}^2 = E(0). \quad (25)$$

5.3.1 Theoretical energy of the Gaussian

Since the initial velocity vanishes, the initial kinetic energy is zero and the theoretical energy is entirely potential.

$$E_{\text{theory}} = \frac{1}{2} \int_{\mathbb{R}^d} |\nabla u_0|^2 d\mathbf{x}. \quad (26)$$

For (24),

$$\nabla u_0 = -\mathbf{x} \exp\left(-\frac{|\mathbf{x}|^2}{2}\right), \quad |\nabla u_0|^2 = |\mathbf{x}|^2 e^{-|\mathbf{x}|^2}.$$

Using the Gaussian integral gives

$$E_{\text{theory}} = \frac{d}{4} \pi^{d/2}. \quad (27)$$

Consequently,

$$E_{\text{theory}}^{2D} = \frac{\pi}{2} = 1.57079632679, \quad E_{\text{theory}}^{3D} = \frac{3}{4} \pi^{3/2} = 4.17624599762. \quad (28)$$

5.3.2 Energy computed by the theta solver

The theta solver stores the displacement and velocity coefficient vectors $\mathbf{U}^n$ and $\mathbf{V}^n$. It computes the kinetic, potential and total energies using the assembled mass and stiffness matrices:

$$\boxed{\begin{aligned} E_{\text{kin},\theta}^n &= \frac{1}{2} (\mathbf{V}^n)^T M \mathbf{V}^n, \\ E_{\text{pot},\theta}^n &= \frac{1}{2} (\mathbf{U}^n)^T A \mathbf{U}^n, \\ E_{\theta}^n &= E_{\text{kin},\theta}^n + E_{\text{pot},\theta}^n. \end{aligned}} \quad (29)$$

For $\theta = 1/2$, the update is Crank–Nicolson. In the homogeneous case, exact algebraic solves preserve (29). A nonzero but constant difference between E_θ^n and E_{theory} is therefore mainly an initial spatial projection error rather than energy lost during time integration.

5.3.3 Natural and staggered energy for matrix-free CG

The matrix-free CG solver stores displacement at three consecutive time levels. The velocity at the integer time t_n is reconstructed by the centred difference

$$\mathbf{V}_{\text{nat}}^n = \frac{\mathbf{U}^{n+1} - \mathbf{U}^{n-1}}{2k}. \quad (30)$$

The natural energy reported in the main plots is

$$\boxed{\begin{aligned} E_{\text{kin,nat}}^n &= \frac{1}{2} \int_{\Omega} \left(\frac{u_h^{n+1} - u_h^{n-1}}{2k} \right)^2 dx, \\ E_{\text{pot,nat}}^n &= \frac{1}{2} \int_{\Omega} |\nabla u_h^n|^2 dx, \\ E_{\text{nat}}^n &= E_{\text{kin,nat}}^n + E_{\text{pot,nat}}^n. \end{aligned}} \quad (31)$$

This is a same-time approximation of the continuous physical energy, but it is not the exact invariant of leapfrog. It may therefore have a bounded $O(k^2)$ variation.

Leapfrog naturally defines the velocity at the half-time level,

$$\mathbf{V}^{n+1/2} = \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{k}.$$

The corresponding staggered CG energy is

$$\boxed{\mathcal{E}_{CG}^{n+1/2} = \frac{1}{2} \left\| \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{k} \right\|_{M_L}^2 + \frac{1}{2} (\mathbf{U}^{n+1})^T A_Q \mathbf{U}^n}. \quad (32)$$

Equivalently, the staggered potential contribution is

$$\frac{1}{2} \int_{\Omega} \nabla u_h^n \cdot \nabla u_h^{n+1} dx.$$

For a symmetric spatial operator and a stable time step, (32) is the discrete quantity conserved by the homogeneous leapfrog recurrence.

5.3.4 Natural and staggered energy for DG/SIPG

The DG solver uses the same centred reconstruction for its natural kinetic energy. In the plotted natural energy, the potential part is computed only from the broken element gradients:

$$\boxed{\begin{aligned} E_{\text{kin,nat,DG}}^n &= \frac{1}{2} \sum_K \int_K \left(\frac{u_h^{n+1} - u_h^{n-1}}{2k} \right)^2 dx, \\ E_{\text{pot,nat,DG}}^n &= \frac{1}{2} \sum_K \int_K |\nabla u_h^n|^2 dx. \end{aligned}} \quad (33)$$

The second expression does not include the consistency, jump and penalty terms of the SIPG form (11). It is useful as an approximation of the physical volume energy, but it is not the full discrete energy of the DG operator.

The solver therefore also computes the complete staggered SIPG energy

$$\mathcal{E}_{DG}^{n+1/2} = \frac{1}{2} \left\| \frac{\mathbf{U}^{n+1} - \mathbf{U}^n}{k} \right\|_{M_L}^2 + \frac{1}{2} a_{DG}(u_h^n, u_h^{n+1}). \quad (34)$$

Here a_{DG} is the full bilinear form in (11), including the volume, interior-face, boundary and penalty contributions. This is the appropriate quantity for assessing conservation of the DG/SIPG leapfrog scheme.

5.3.5 Error used in the analysis

The graphs normalize the kinetic, potential and total natural energy by E_{theory} . Their right-hand panels display the absolute relative error

$$\varepsilon_{\text{theory}}(t) = \frac{|E_{\text{nat}}(t) - E_{\text{theory}}|}{|E_{\text{theory}}|}. \quad (35)$$

This diagnostic measures agreement with the continuous Gaussian energy. It includes spatial approximation, quadrature, initialization and time-discretization effects.

Discrete conservation should instead be assessed using

$$\varepsilon_{\text{drift}}(t) = \frac{|E_{\text{disc}}(t) - E_{\text{disc}}(0)|}{|E_{\text{disc}}(0)|}, \quad (36)$$

where E_{disc} is the natural matrix energy (29) for Crank–Nicolson and the staggered energy (32) or (34) for the explicit solvers.

5.3.6 Summary of the numerical results

Table 1 reports the most significant values obtained from the supplied natural-energy plots. The error values are rounded readings from the graphs and are therefore denoted by $\approx$. They compare the natural energy with the continuous Gaussian reference and should not be interpreted as measurements of staggered-energy drift.

Solver	Dimension	Samples	E_{theory}	$\max_t \varepsilon_{\text{theory}}$	$\varepsilon_{\text{theory}}(T)$
Crank–Nicolson	2D	4	1.570796327	$\approx 6.8 \times 10^{-9}$	$\approx 6.8 \times 10^{-9}$
Crank–Nicolson	3D	2	4.176245998	$\approx 5.6 \times 10^{-4}$	$\approx 5.6 \times 10^{-4}$
Matrix-free CG	2D	85	1.570796327	$\approx 6.9 \times 10^{-4}$	$\approx 1.3 \times 10^{-4}$
Matrix-free CG	3D	21	4.176245998	$\approx 7.5 \times 10^{-4}$	$\approx 4.5 \times 10^{-4}$
Matrix-free DG/SIPG	2D	683	1.570796327	$\approx 2.3 \times 10^{-4}$	$\approx 2.5 \times 10^{-5}$
Matrix-free DG/SIPG	3D	171	4.176245998	$\approx 3.1 \times 10^{-3}$	$\approx 5.0 \times 10^{-5}$

Table 1: Natural-energy error relative to the analytical energy of the unit-width Gaussian.

The Crank–Nicolson energy remains constant throughout both runs. The approximately 6.8×10^{-9} relative error in two dimensions shows that the projected Gaussian energy is extremely close to its analytical value. The larger but constant three-dimensional error, approximately 5.6×10^{-4} , is consistent with the coarser 3D spatial approximation. Its constancy indicates that it is not an accumulating time-integration error.

The matrix-free CG natural energy exhibits a smooth and bounded variation. This agrees with the fact that the natural energy is only an $O(k^2)$ approximation of the leapfrog invariant.

A zero or small value of (35) at one instant means only that the numerical energy crosses the theoretical value; it does not mean that the full numerical solution is exact.

The DG natural-energy error is more oscillatory. Part of the DG energy is stored in the SIPG jump and penalty contributions, which are absent from (33). The larger temporary 3D error therefore does not by itself demonstrate instability. The error remains bounded and shows no systematic growth. Plotting the drift of (34) would provide the more direct DG conservation test.

The numbers of recorded samples must not be used as accuracy indicators. The explicit methods use CFL-restricted time steps, and the DG method uses the smallest step. Moreover, the two- and three-dimensional runs use different spatial refinement levels. A strict ranking of the solvers would require matched mesh resolution, time step, polynomial degree or computational cost.

Finally, the explicit benchmark initializes the previous displacement as

$$\mathbf{U}^{-1} = \mathbf{U}^0 - k\mathbf{V}^0.$$

A second-order initialization would include the acceleration,

$$\mathbf{U}^{-1} = \mathbf{U}^0 - k\mathbf{V}^0 + \frac{k^2}{2}\mathbf{a}^0, \quad \mathbf{a}^0 = M_L^{-1}(\mathbf{F}^0 - A_*\mathbf{U}^0). \quad (37)$$

Although the initial velocity is zero, the Gaussian has nonzero initial acceleration. Omitting the acceleration term introduces an $O(k^2)$ startup contribution to the natural-energy error of the CG and DG runs.

5.4 Convergence analysis using a manufactured solution

The method of manufactured solutions is used to verify the complete space–time implementation. A smooth function is prescribed as the exact solution and substituted into the differential equation. The resulting residual defines a forcing term for which the exact solution is known. The numerical solution can therefore be compared directly with the prescribed function, and a sequence of uniformly refined meshes can be used to measure the experimental order of convergence.

5.4.1 Manufactured problem

The test is performed in two dimensions on $\Omega = (0, 1)^2$ up to the final time $T = 1$. The function implemented in `WaveFunctions.hpp` is

$$u_{\text{ex}}(x, y, t) = t^3 \sin(\pi x) \sin(\pi y). \quad (38)$$

Its first and second time derivatives and its Laplacian are

$$\partial_t u_{\text{ex}} = 3t^2 \sin(\pi x) \sin(\pi y), \quad (39)$$

$$\partial_{tt} u_{\text{ex}} = 6t \sin(\pi x) \sin(\pi y), \quad (40)$$

$$\Delta u_{\text{ex}} = -2\pi^2 t^3 \sin(\pi x) \sin(\pi y). \quad (41)$$

Consequently, substitution into $u_{tt} - \Delta u = f$ gives the manufactured forcing

$$f(x, y, t) = (6t + 2\pi^2 t^3) \sin(\pi x) \sin(\pi y). \quad (42)$$

Because the sine factors vanish on every side of the unit square, (38) satisfies the homogeneous Dirichlet boundary condition. Moreover,

$$u_{\text{ex}}(x, y, 0) = 0, \quad \partial_t u_{\text{ex}}(x, y, 0) = 0, \quad (43)$$

so the zero displacement and zero velocity used by the convergence driver are consistent with the exact solution.

5.4.2 Discretizations and refinement procedure

The same manufactured problem is solved with all the three methods. All methods use polynomial degree $p = 4$. The continuous solvers use Q_4 elements, while the DG solver uses discontinuous Q_4 elements with Gauss–Lobatto nodes. For every solver, the unit square is refined globally at levels $\ell = 5, 6, 7$. Thus,

$$h_\ell = 2^{-\ell} \in \left\{ \frac{1}{32}, \frac{1}{64}, \frac{1}{128} \right\}. \quad (44)$$

The corresponding numbers of degrees of freedom are 16641, 66049, and 263169 for both continuous methods, and 25600, 102400, and 409600 for DG.

The time step is coupled to the mesh size. The theta solver initially takes $k = h_{\text{diam}}/2$ and then adjusts it so that the final time is reached exactly. The matrix-free solvers use CFL-type choices

$$k_{\text{CG}} = \frac{0.1}{p}h, \quad k_{\text{DG}} = \frac{0.05}{p^2}h, \quad (45)$$

At the end of each run, the displacement is compared with $u_{\text{ex}}(\cdot, T)$. The reported errors are

$$e_{L^2}(h) = \left(\int_{\Omega} |u_h - u_{\text{ex}}|^2 d\mathbf{x} \right)^{1/2} \quad (46)$$

and

$$e_{H^1}(h) = \left(e_{L^2}(h)^2 + \sum_{K \in \mathcal{T}_h} \int_K |\nabla u_h - \nabla u_{\text{ex}}|^2 d\mathbf{x} \right)^{1/2}. \quad (47)$$

For the continuous methods, (47) is the usual H^1 error.

Because every refinement halves h , the experimental order associated with two consecutive errors is

$$r_i = \frac{\log(e_i/e_{i+1})}{\log(h_i/h_{i+1})} = \log_2 \left(\frac{e_i}{e_{i+1}} \right). \quad (48)$$

5.4.3 Numerical results

Table 2 reports the displacement errors. A dash indicates that no rate can be computed on the first mesh.

Table 2: Manufactured-solution errors at $T = 1$ for polynomial degree $p = 4$.

Solver	h	e_{L^2}	r_{L^2}	e_{H^1}	r_{H^1}
Theta	1/32	2.56080×10^{-5}	–	1.16620×10^{-4}	–
	1/64	6.54816×10^{-6}	1.97	2.98211×10^{-5}	1.97
	1/128	1.63734×10^{-6}	2.00	7.52491×10^{-6}	1.99
Matrix-free CG	1/32	6.62093×10^{-8}	–	3.04286×10^{-7}	–
	1/64	1.65521×10^{-8}	2.00	7.54223×10^{-8}	2.01
	1/128	4.13757×10^{-9}	2.00	1.88433×10^{-8}	2.00
Matrix-free DG/SIPG	1/32	1.03372×10^{-9}	–	4.43119×10^{-8}	–
	1/64	2.49364×10^{-10}	2.05	2.85353×10^{-9}	3.96
	1/128	2.54216×10^{-11}	3.29	3.08171×10^{-10}	3.21

The theta solver exhibits a clear second-order trend in both norms. Its rates approach 2 as the mesh is refined, in agreement with the temporal accuracy of the Crank–Nicolson method when $k = O(h)$. The matrix-free CG solver also gives essentially exact second-order rates: the two measured L^2 rates are 2.000 and 2.000, while the H^1 rates are 2.012 and 2.001. In this

experiment, the second-order temporal and startup errors therefore dominate the higher-order spatial approximation.

The DG solver produces the smallest absolute errors at every tested mesh level. At $h = 1/128$, its L^2 error is 2.54×10^{-11} and its broken H^1 error is 3.08×10^{-10} . Its observed rates, however, vary from 2.05 to 3.29 in L^2 and from 3.96 to 3.21 in the H^1 norm. These two ratios do not establish a single asymptotic order. The much smaller DG time step suppresses part of the temporal error, so the measured values reflect a changing combination of spatial, temporal, quadrature, and roundoff contributions. Additional refinement levels, preferably together with an independent time-step study, are needed before assigning an asymptotic DG rate.

Overall, the results verify that all three implementations converge toward the manufactured solution. Theta and matrix-free CG demonstrate robust second-order fully discrete convergence. DG/SIPG attains substantially smaller errors for the chosen parameters.

6 Numerical dispersion analysis

6.1 Purpose and experimental configuration

Numerical dispersion is the error introduced by discretization in the frequency and propagation speed of waves. For the homogeneous wave equation

$$\frac{\partial^2 u}{\partial t^2} - c^2 \Delta u = 0, \quad (49)$$

a plane wave $u = e^{i(\boldsymbol{\kappa} \cdot \mathbf{x} - \omega t)}$ satisfies

$$\omega = c|\boldsymbol{\kappa}|. \quad (50)$$

Its phase speed is therefore independent of wavelength. Spatial and temporal discretization modify this relation, potentially producing phase shifts and deformation of a wave packet.

Dispersion analysis complements energy monitoring: a method may preserve discrete energy while producing inaccurate phases. Similarly, a reduction in peak amplitude may result from wave-packet spreading rather than energy dissipation.

The implemented experiment considers the undamped, unforced wave equation on

$$\Omega = [-15, 15]^2, \quad u = 0 \quad \text{on } \partial\Omega, \quad (51)$$

with a Gaussian-modulated sinusoidal initial displacement:

$$u_0(x, y) = G(x - x_0)S(y), \quad (52)$$

where

$$G(\xi) = \exp\left(-\frac{\xi^2}{2\sigma^2}\right) \cos(\kappa\xi), \quad S(y) = \sin\left(\frac{\pi(y - y_{\min})}{L_y}\right). \quad (53)$$

The initial velocity is

$$v_0(x, y) = -c G'(x - x_0)S(y). \quad (54)$$

The parameters are

$$\begin{aligned} \kappa &= 2\pi, & \lambda &= \frac{2\pi}{\kappa} = 1, & \sigma &= 1, \\ x_0 &= -8, & c &= 1, & T &= 18, & y_{\min} &= -15, & L_y &= 30. \end{aligned} \quad (55)$$

Here κ denotes the carrier wavenumber, σ the Gaussian envelope width.

6.2 Reference solution and peak measurement

The code compares the numerical displacement with the translating reference

$$u_{\text{ref}}(x, y, t) = G(x - x_0 - c_{\text{eff}}t)S(y), \quad c_{\text{eff}} = \sqrt{1 + \frac{\pi^2}{\kappa^2 L_y^2}}, \quad (56)$$

where the expression for c_{eff} corresponds to the implemented choice $c = 1$.

Along the centreline $y = 0$, the reference maximum occurs at

$$x_{\text{ref}}^* = x_0 + c_{\text{eff}}T \approx 10.00250. \quad (57)$$

Thus, the reference peak is calculated directly rather than located by a numerical search. The code's introductory message prints $x_0 + cT = 10$, whereas the diagnostics use the corrected value in (57). These operations are implemented in `measure_dispersion()` in `dispersion_analysis.hpp`.

The numerical peak is estimated along $y = 0$ within

$$[x_{\text{ref}}^* - 2.5, x_{\text{ref}}^* + 2.5]. \quad (58)$$

The routine first samples 500 points and selects the largest positive crest. It then performs finer local sampling and, when its acceptance checks are satisfied, applies a three-point parabolic correction:

$$x_h^* \approx x_m - \frac{\delta}{2} \frac{u_+ - u_-}{u_+ - 2u_m + u_-}, \quad (59)$$

where u_- , u_m and u_+ are values at equally spaced locations $x_m - \delta$, x_m and $x_m + \delta$ around the candidate maximum. This procedure is implemented in `locate_peak_1d()` in `WaveSolverBase.hpp`.

The routine detects a displacement crest, not the envelope directly. If the packet is strongly distorted, a neighbouring crest may become the largest. The resulting shift must therefore be checked against waveform plots.

6.3 Diagnostic quantities

The measured peak displacement is converted into an equivalent delay, a phase indicator and a relative delay:

$$\begin{aligned} \Delta x &= x_{\text{ref}}^* - x_h^*, & \Delta t &= \frac{\Delta x}{c_{\text{eff}}}, \\ \Delta \phi &= \kappa \Delta x, & r_t &= \frac{\Delta t}{T}. \end{aligned} \quad (60)$$

Positive values indicate a detected peak behind the reference; negative values indicate a peak ahead.

Two absolute displacement errors are also evaluated:

$$e_{\text{raw}} = \|u_h(\cdot, T) - u_{\text{ref}}(\cdot, T)\|_{L^2(\Omega)}, \quad (61)$$

and

$$e_{\text{aligned}} = \|u_h(\cdot, T) - u_{\text{ref}}(\cdot, T - \Delta t)\|_{L^2(\Omega)}. \quad (62)$$

The second diagnostic compares the numerical field with a reference translated to match the detected peak.

A reduction after alignment indicates that translation contributes to the discrepancy. The remaining error includes shape, amplitude and reference-model differences.

6.4 Polynomial-degree study

The first experiment compares the continuous Galerkin theta solver, with Crank–Nicolson parameter $\theta = 1/2$, for degrees $p = 1, 2, 4, 6$. Mesh refinement is adjusted to obtain approximately comparable DoF counts. The supplied results are reported in Table 3.

p	Refinement	DoFs	e_{raw}	e_{aligned}	Δt	$\Delta\phi$
1	7	16641	5.506	5.289	-1.351	-8.489
2	6	16641	6.203	2.686	-1.518	-9.538
4	5	16641	4.680	2.536	-1.266	-7.957
6	4	9409	5.608	3.895	-1.672	-10.504

Table 3: Reported polynomial-degree study for the theta solver. Phase indicators are expressed in radians.

All detected peaks lie ahead of the reference. However, these measurements do not prove that consistent-mass discretizations always produce a phase lead: the fully discrete behaviour also depends on time integration and on the reliability of crest identification.

Alignment reduces the error by approximately 4%, 57%, 46% and 31%, respectively. Translation therefore explains an appreciable part of the discrepancy for some configurations, but the remaining errors are substantial.

Among these configurations, $p = 4$ gives the smallest aligned error and delay magnitude. The poorer $p = 6$ result cannot be attributed to degree alone because that configuration has fewer DoFs and a coarser mesh.

6.5 Matrix-free CG and DG comparison

The second experiment uses degree $p = 4$ and the same mesh of $32 \times 32 = 1024$ cells. CG has 16641 DoFs, while DG has 25600 because neighbouring elements do not share their unknowns. Table 4 reports the supplied results.

Solver	e_{raw}	e_{aligned}	Δt	$\Delta\phi$	r_t (%)
Matrix-free CG	1.042	0.8530	+0.07638	+0.4799	0.424
Matrix-free DG/SIPG	1.358	0.8874	+0.12248	+0.7696	0.680

Table 4: Reported matrix-free CG and DG comparison at $p = 4$. Phase indicators are expressed in radians, and relative delays are expressed as percentages.

Both detected peaks lag behind the reference. CG has the smaller delay, raw error and aligned error in this experiment. Alignment reduces the error by approximately 18% for CG and 35% for DG, indicating that peak displacement explains only part of the discrepancy.

The different delays cannot be attributed solely to the SIPG penalty. The methods differ in continuity, spatial operators, DoF counts and time-step choices. Likewise, Gauss–Lobatto mass lumping does not establish a universal sign for the fully discrete phase error.

7 Benchmark performance comparison

7.1 Experimental configuration and timing definition

The supplied benchmark results compare the assembled continuous Galerkin theta solver, with the default Crank–Nicolson parameter $\theta = 1/2$, the matrix-free continuous Galerkin solver,

and the matrix-free DG/SIPG solver. All three methods use polynomial degree $p = 4$ and integrate the unforced wave equation on $\Omega = [-15, 15]^d$ up to $T = 1$, starting from the Gaussian displacement and zero velocity specified by the benchmark driver.

The two-dimensional tests use refinement level $\ell = 6$, giving $64 \times 64 = 4096$ cells with side length $h = 0.46875$. The three-dimensional tests use $\ell = 4$, giving $16^3 = 4096$ cells with $h = 1.875$. Within each dimension, the solvers therefore use the same mesh, but DG has more degrees of freedom because its unknowns are not shared between neighbouring elements. Every log reports one MPI rank and eight threads per rank. This records the configured execution environment; it does not demonstrate that every solver uses all eight threads equally.

The quantity reported as *compute time* is the accumulated wall-clock time of the timed operations inside the time loop. Mesh generation, solver setup, initial-condition construction and visualization output are excluded. In particular, the initial assembly of the theta mass and stiffness matrices, and the matrix-free setup and inverse-mass construction, are outside these timers. The theta timer also excludes the final copies into its history vectors, whereas the matrix-free timers include their vector swaps.

7.2 Measured performance

Let t_{comp} denote the reported compute time and N_t the reported number of steps. Define

$$\bar{t}_{\text{step}} = \frac{t_{\text{comp}}}{N_t}, \quad S_\theta = \frac{t_{\text{comp},\theta}}{t_{\text{comp},\text{solver}}}. \quad (63)$$

The speedup S_θ compares runs within the same dimension; values greater than one indicate a shorter compute time than theta. Table 5 reproduces the logged DoFs, step counts and timings, and adds this derived speedup.

Dim.	Solver	DoFs	N_t	t_{comp} (s)	$\bar{t}_{\text{step}}$ (s)	S_θ
2D	Theta	66049	4	0.8585	0.2146	1.00
2D	CG	66049	85	0.1597	0.0019	5.38
2D	DG/SIPG	102400	682	4.7059	0.0069	0.18
3D	Theta	274625	1	42.9373	42.9373	1.00
3D	CG	274625	21	0.2302	0.0110	186.52
3D	DG/SIPG	512000	170	13.2722	0.0781	3.24

Table 5: Performance from the six supplied benchmark logs, with $p = 4$, $T = 1$, 4096 cells, one MPI rank and eight configured threads per rank. MF denotes matrix-free. Timings retain the precision of the logs; speedups are computed from the reported compute times. The runs use different time steps and are not matched for accuracy.

Matrix-free CG is the fastest solver in both experiments. In 2D, its compute time is 0.1597 s, compared with 0.8585 s for theta and 4.7059 s for DG. It is therefore approximately 5.38 times faster than theta and 29.47 times faster than DG. In 3D, the corresponding factors increase to 186.52 and 57.66. DG is approximately 5.48 times slower than theta in 2D, but 3.24 times faster than theta in 3D.

7.3 Interpretation of the runtime differences

Total compute time combines the number of updates and their cost. Before adjustment to reach T exactly, the explicit solvers choose

$$k_{\text{CG},0} = \frac{0.1}{p}h = 0.025h, \quad k_{\text{DG},0} = \frac{0.05}{p^2}h = 0.003125h. \quad (64)$$

The prescribed DG step is eight times smaller. The logged steps are approximately 0.0117647 and 0.00146628 in 2D, and 0.047619 and 0.00588235 in 3D, for CG and DG respectively. Consequently, DG performs about eight times as many updates. The theta solver starts from half the minimum cell diameter and adjusts its step to T , giving $k = 0.25$ in 2D and $k = 1$ in 3D.

DG also has a higher cost per update. Computing the ratios from t_{comp}/N_t , before rounding the average times, gives DG-to-CG factors of approximately 3.67 in 2D and 7.12 in 3D. Together with the step-count ratios, these account for the total runtime factors of 29.47 and 57.66. The DG implementation evaluates volume, interior-face and boundary-face contributions, then applies the inverse mass diagonal. Its DoF count exceeds CG by approximately 55.0% in 2D and 86.4% in 3D. These additional unknowns and face operations provide an implementation-based explanation for the greater update cost, although the logs do not separate their individual contributions.

The theta method takes fewer steps, but each step constructs right-hand sides, copies and combines sparse matrices, applies boundary conditions, initializes Jacobi preconditioners and solves two systems using conjugate gradients. Its loop also assembles the forcing vectors at two time levels even for this unforced benchmark. Matrix-free CG instead applies the explicit cell operator and the inverse diagonal mass without iterative linear solves. The observed timings are consistent with this difference in computational work.

8 Parallel performance and scaling on MeluXina

The preceding benchmark comparison is extended here with a separate performance campaign on the MeluXina CPU partition. The experiments compare matrix-free continuous Galerkin (CG) and the assembled theta solver through strong scaling, weak scaling and mesh-dependent solver comparisons.

8.1 Computing environment and numerical configuration

- **Processor architecture:** Dual-socket AMD EPYC 7H12 processors, based on the Zen 2 microarchitecture and a 7 nm manufacturing process.
- **Core topology:** 64 physical cores per socket, totaling **128 physical cores per compute node**. Simultaneous Multithreading (SMT) supports up to 256 hardware threads per node. The base clock frequency is 2.60 GHz, with a maximum boost frequency of 3.30 GHz.
- **Cache subsystem:**
 - **L1 data cache:** 32 KB per core, private and 8-way set-associative.
 - **L2 cache:** 512 KB per core, private and 8-way set-associative.
 - **L3 cache:** 256 MB per socket, organized into 16 MB slices per Core Complex (CCX), totaling **512 MB per dual-socket node**.
- **Memory subsystem:** 512 GB of DDR4-3200 ECC memory, distributed across eight memory channels per socket and sixteen channels per node, with a theoretical peak aggregate memory bandwidth of approximately **410 GB/s**.
- **Interconnect:** Mellanox ConnectX-6 HDR InfiniBand, operating at **200 Gbps per link** in a non-blocking fat-tree topology, with Unified Communication X (UCX) support for accelerated point-to-point communication.

Throughout this section, P denotes MPI process count, $p = 4$ polynomial degree, ℓ refinement level, N global DoFs and N_t time steps. The plots label their horizontal axes as processes/cores.

We use process count because the effective number of threads and their CPU placement are part of the reported execution issue. Compute time, t_{comp} , measures the timed time-stepping operations and excludes mesh generation, initial setup and visualization output. It is not the complete elapsed application time.

8.2 Definitions of the performance metrics

For strong scaling, the numerical workload of each solver is fixed and its own one-process time is the reference:

$$S_s(P) = \frac{t_{\text{comp}}(1)}{t_{\text{comp}}(P)}, \quad E_s(P) = \frac{S_s(P)}{P} 100\%. \quad (65)$$

Ideal linear scaling gives $S_s = P$ and $E_s = 100\%$. Values above 100% describe superlinear speedup relative to the measured baseline; they do not identify its cause.

For weak scaling, the notes use a spatial-size normalization:

$$S_{w,N}(P) = \frac{t_{\text{comp}}(1)}{t_{\text{comp}}(P)} \frac{N(P)}{N(1)}, \quad E_{w,N}(P) = \frac{S_{w,N}(P)}{P} 100\%. \quad (66)$$

Maintaining N/P approximately constant preserves spatial work per rank, but not total work if refinement changes N_t . To distinguish these effects, define average step cost and DoF-update throughput by

$$\tau(P) = \frac{t_{\text{comp}}(P)}{N_t(P)}, \quad Q(P) = \frac{N(P)N_t(P)}{10^6 t_{\text{comp}}(P)} \quad [\text{MDoF} \cdot \text{steps/s}]. \quad (67)$$

The corresponding throughput-normalized efficiency is

$$E_Q(P) = \frac{Q(P)}{PQ(1)} 100\% = E_{w,N}(P) \frac{N_t(P)}{N_t(1)}. \quad (68)$$

Throughput counts discrete updates rather than physical simulated time or floating-point operations. It is useful for comparing the update rate within a solver, but an implicit step and an explicit step do different work and need not achieve the same accuracy.

8.3 Strong scaling for 2D problem

The fixed mesh has $\ell = 7$, 16384 quadrilateral cells and $N = (4 \times 128 + 1)^2 = 263169$ DoFs. The process sweep is $P \in \{1, 2, 4, 8, 16, 32, 64\}$. CG performs 85 steps and theta performs three, with these counts held fixed across the strong-scaling sweep. Table 6 reproduces the reported compute times and recomputes speedups and efficiencies from them.

P	CG			Theta		
	t_{comp} (s)	S_s	E_s (%)	t_{comp} (s)	S_s	E_s (%)
1	0.2832	1.0000	100.00	2.4974	1.0000	100.00
2	0.1126	2.5151	125.75	1.0045	2.4862	124.31
4	0.0538	5.2639	131.60	0.3997	6.2482	156.20
8	0.0305	9.2852	116.07	0.1933	12.9198	161.50
16	0.0293	9.6655	60.41	0.1582	15.7863	98.66
32	54.0907	0.00524	0.0164	28.8373	0.08660	0.2706
64	219.8171	0.00129	0.0020	99.5589	0.02508	0.0392

Table 6: 2D strong-scaling results on MeluXina.

From one to eight processes, both methods show superlinear speedup at the sampled points. CG reaches its maximum efficiency at four processes, approximately 131.6%, while theta reaches approximately 161.5% at eight. At sixteen processes, theta still retains about 98.7% efficiency, whereas CG falls to 60.4%. The smallest recorded compute times are nevertheless obtained at sixteen processes for both solvers: 0.0293 s for CG and 0.1582 s for theta. CG is approximately 5.40 times faster at that point. The CG improvement from eight to sixteen processes is only about 4%, suggesting diminishing returns for this workload. At 32 and 64 processes, both times increase by orders of magnitude. The observed slowdown is clear. Figure 1 as evidence of the problematic execution configuration.

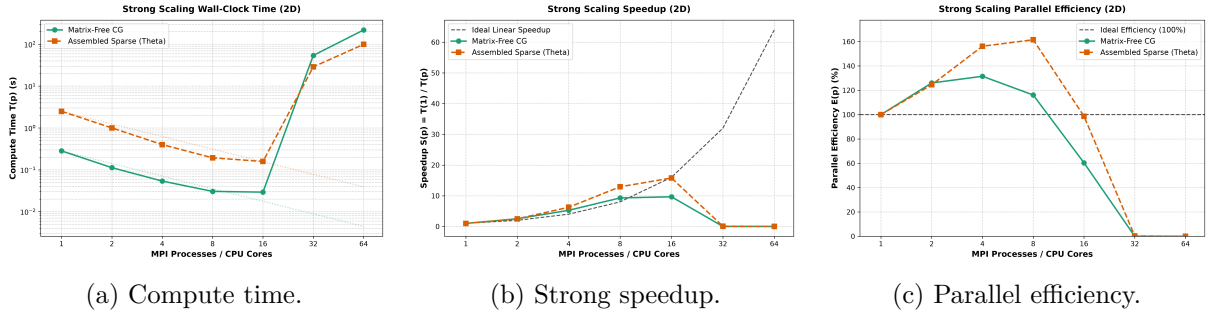


Figure 1: Strong scaling in two spatial dimensions: compute time, speedup and parallel efficiency.

8.4 Weak scaling in two spatial dimensions

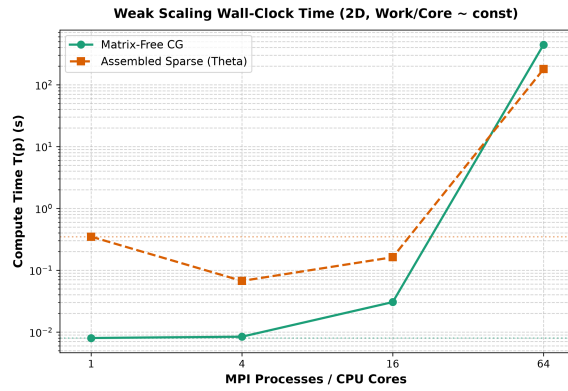
The weak-scaling mesh sequence increases the process count by four per refinement. Spatial work per rank stays near 16500 DoFs, while both solvers increase their step counts. The counts must be distinguished: the 21, 43, 85, 171 sequence belongs to CG, whereas the theta counts are 1, 2, 3, 6, consistent with its reported total and per-step times.

P	ℓ	N	N/P	$N_{t,CG}$	$N_{t,\theta}$
1	5	16641	16641.00	21	1
4	6	66049	16512.25	43	2
16	7	263169	16448.06	85	3
64	8	1050625	16416.02	171	6

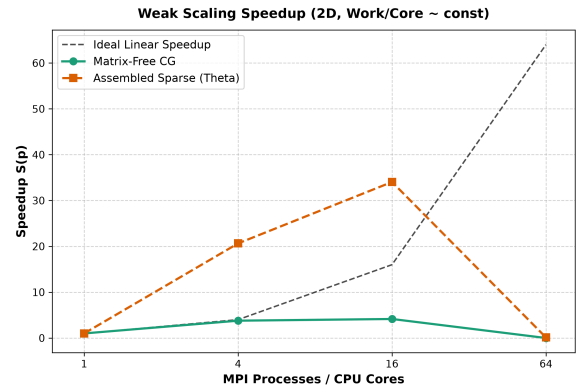
Table 7: 2D weak-scaling workloads. Constant spatial workload per rank does not imply constant total workload over the simulation interval.

P	Solver	t_{comp} (s)	τ (ms)	Q	$E_{w,N}$ (%)
1	CG	0.0080	0.381	43.683	100.00
4	CG	0.0084	0.195	338.108	94.50
16	CG	0.0305	0.359	733.422	25.93
64 [†]	CG	445.0086	2602.389	0.404	0.0018
1	Theta	0.3489	348.900	0.0477	100.00
4	Theta	0.0671	33.550	1.969	515.95
16	Theta	0.1621	54.033	4.870	212.74
64 [†]	Theta	179.2123	29868.717	0.0352	0.1921

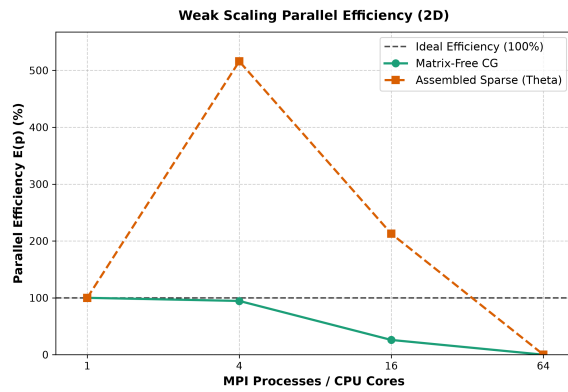
Table 8: 2D weak-scaling measurements and derived metrics. Q is in MDoF·steps/s. [†]: reported oversubscription. The strong and weak sweeps are separate measurements, including where their mesh and process counts coincide.



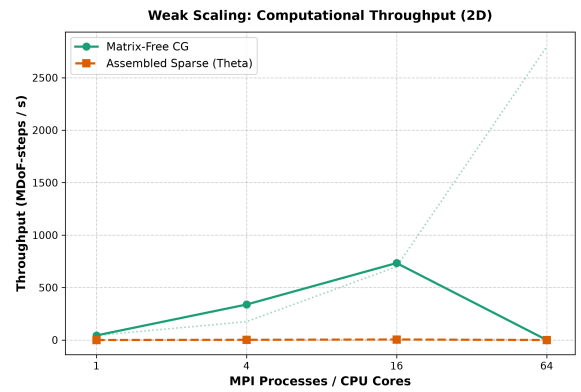
(a) Compute time.



(b) Spatially normalized speedup.



(c) Spatially normalized efficiency.



(d) Computational throughput.

Figure 2: Weak scaling in two spatial dimensions: compute time, speedup, efficiency and throughput.

8.5 Scaling results and solver comparison

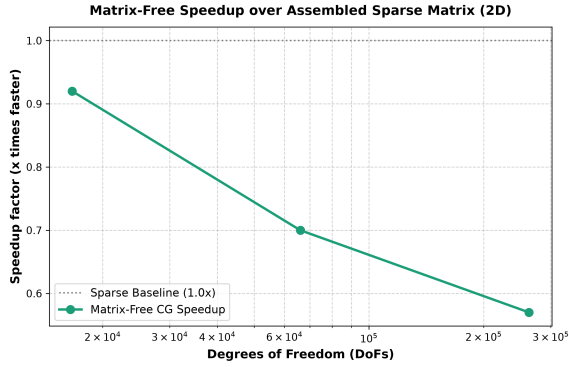
Weak-scaling behaviour. The weak-scaling experiments increase the mesh size and MPI process count together, keeping the spatial workload per process approximately constant. However, mesh refinement also increases the number of time steps required by the explicit CG solver. Consequently, an increase in total compute time does not necessarily indicate poor parallel scaling: each process must also perform more temporal updates.

CG maintains similar average step costs at one and sixteen processes, with a lower cost at four processes. Theta also benefits from parallel execution, completing the four-process run faster than its one-process baseline despite the larger problem. This produces superlinear spatially normalized speedup, although the timings alone do not establish whether cache behaviour, solver iterations or execution variability explains the improvement.

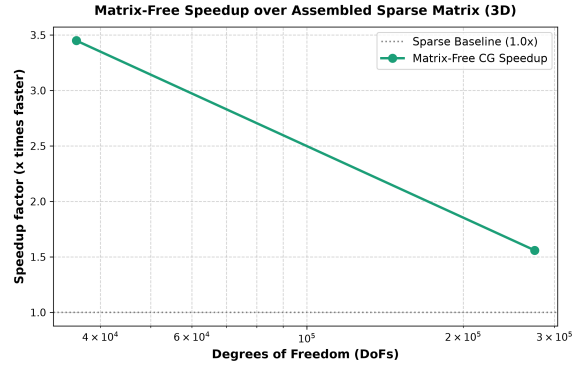
At sixteen processes, CG completes the weak-scaling simulation approximately 5.3 times faster than theta. Its advantage comes from cheaper explicit updates, even though it requires more time steps. This total-time comparison is more directly relevant to simulation performance than the much larger difference in update throughput.

Direct comparison in 2D and 3D. A separate experiment compares the solvers at a fixed allocation of 32 MPI processes while increasing mesh refinement. In 2D, theta achieves lower total compute time on all tested meshes. Although individual CG steps are substantially cheaper, the explicit method requires enough additional updates to outweigh this advantage. The difference becomes more pronounced on the finer meshes.

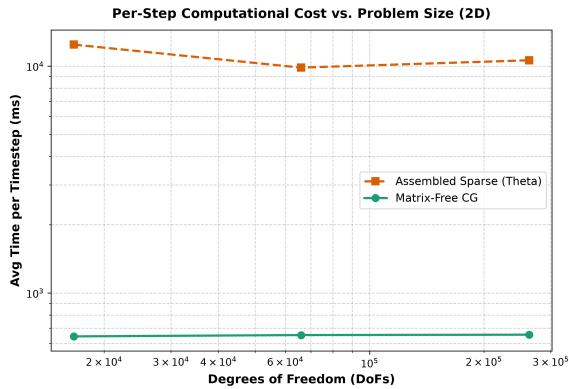
In 3D, CG completes the recorded simulations approximately 3.46 and 1.56 times faster than theta on the coarse and fine meshes, respectively. Its advantage decreases with refinement as the number of explicit steps increases. These results show that total runtime depends on both the cost of an update and the number of updates required. The different rankings in 2D and 3D therefore cannot be attributed to spatial dimension alone.



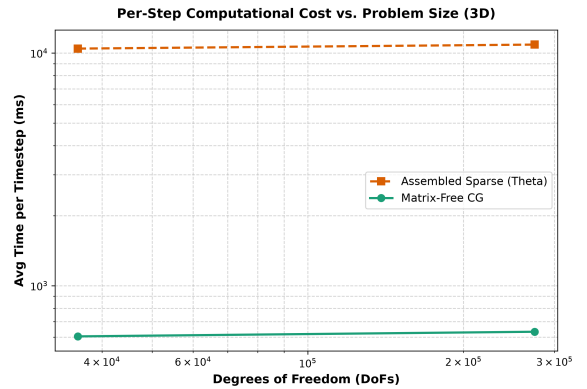
(a) Theta is faster overall: the ratio stays below 1 and decreases with refinement.



(b) CG is faster overall, with its advantage decreasing from approximately 3.46 to 1.56.



(c) CG has cheaper individual steps despite its longer total runtime in 2D.



(d) CG maintains a lower per-step cost in 3D.

Figure 3: Comparison of overall speedup (top) and per-step computational cost (bottom) across 2D (left) and 3D (right) configurations.